

School of Industrial and Information Engineering

Master of Science in Information Engineering
Master of Science in Biomedical Engineering

POLITECNICO DI MILANO

Dipartimento di
Elettronica, Informazione
e Bioingegneria







Practice on introduction to R scripting:
R and Bioconductor

Silvia Cascianelli *silvia.cascianelli@polimi.it*

Introduction to R

- R is an **open, cross-platform, software environment** developed for **statistical computing, data analysis and graphical representation**.

- R provides a wide variety of techniques, including **linear and nonlinear modeling**, **classical statistical tests**, **time-series analysis**, **classification**, **clustering**, and others.

- R is highly **extensible**.

- R is an interpreted language. The source code for the R environment is written primarily in **C**, **Fortran**, and **R**.

- Downloading R software: **https://www.r-project.org/**

R – Bioconductor

- Bioconductor is a widely used **open source** and **open development** software project.
- The Bioconductor project aims to provide access to a wide range of **powerful statistical, processing and graphical methods for the analysis of genomic data**.
- Most Bioconductor components are distributed as peer-reviewed R packages, which are add-on modules for R.
- Contributions are also written in other programming languages.

R basics – Packages downloading

- Download & install R packages

```
> install.packages("package_name")
```

- Download & install Bioconductor packages

```
> BiocManager::install(c("package_name_1", "package_name_2"))
```

- Load or attach an already installed package

```
> package_name::xxx
> library(package_name)
# load or attach every non built-in R library before using it
```

- .R extension for R code
- .RData extension for R sessions (with multiple objects)
- .Rds extension for storing a ‘single’ R object
- .Rmd extension for R notebooks

R basics – Getting started

- Getting the current working directory

```
> getwd()
```

- Setting the working directory

```
> setwd("/path/directory")
```

- List every object in the current workspace/environment

```
> ls()
```

- Remove object x from the current workspace/environment

```
> rm(x)
```

- Remove multiple objects

```
> rm(x, y, …)
```

- Remove every object in the current workspace/environment

```
> rm(list=ls())
```

R basics – Help

- Online help

```
> help("function")          # help
> ??"..."                   # search by term
```

- List of R demos in package ‘base’

```
> demo()
```

- Getting examples from some packages

```
> example(package_name+tab)
```

- List of datasets in package ‘dataset’

```
> data()
```

- Quit R interpreter

```
> q()
```

R basics – Basic data types and structures

- **Numeric, Integer, double**
- **Character (string)**
- **Logical (TRUE=1, FALSE=0)**
- **Factor (nominal/categorical variable)**

Assign values to variables using the <- operator

Content:	1-D	2-D	n-D
Homogeneous	**Vector**	**Matrix**	**Array**
Heterogeneous	**List**	**Data frame**	

R basics – Basic data structures

- Data type investigation:

```
> typeof(x)
# returns the data type of x
# is.type(x) returns TRUE if x is of type type,
# FALSE otherwise. E.g.:
> is.character(x)

> length(x) #for vectors, lists
> dim(x) #for matrices, arrays, data frames
```

- Common data type conversions:

  as.numeric(x), as.logical (x)

  as.vector(x), as.matrix(x), as.list(x), as.data.frame(x)

  as.factor(x) #returns the number of unique values/levels

R basics – Basic commands

- Scalar assignment

```
> x <- 3
# equivalent to …
> x = 3
```

- Vector assignment

```
# Vectors in R are built with the concatenate
# c() function. They must be homogeneous.
> w <- c(1,2,3)
> v <- c("A","B","C")
> v                    # to return an object in R,
[1] "A" "B" "C"        # simply type its name
```

R basics – Vector indexing

```
> v[i]          # i-th element of v
# from a-th
> v[a:b]        # …to b-th element
> v[a:length(v)-1] # …to last but one element
> v[-i]         # remove the i-th element of v
> v[-a:-b]      # remove from the a-th to the
                # b-th element of v
# NOTE!! Indexes in R start from 1, not 0!!!
```

- Named Vectors

```
> v <- c(first='a', second='b', third='c')
> v['first']
first
  "a"
```

R basics – Vectors: unary operations

```
> v <- c(4,6,4,8,-7,2,3)
```

```
> length(v)
[1] 7
> min(v)
[1] -7
> mean(v)
[1] 2.857143
> var(v)
[1] 22.80952
```

```
> sum(v)
[1] 20
> max(v)
[1] 8
> median(v)
[1] 4
> sd(v)
[1] 4.775932
```

R basics – Vectors: binary operations

```
> v + u     # sum v and u element by element
> v - u     # subtract v and u element by element
> v * u     # multiply v and u element by element
> v / u     # divide v and u element by element
> v %% u    # modulus between v and u elements
```

PLEASE NOTE!! If two vectors have different lengths, R does not return any error! The shorter vector is repeated to compensate for the difference in length. Unfortunately, this rule is still valid for other R data structures.

```
> append(v, u)     # append u to vector v
```

R basics – Vectors: binary comparison operators

```
> v < u # or v > u
> v == u # or v != u
# return Logical vectors with the result of
# the proposition for each (v[i],u[i])
# Control functions in R vectors:
> ifelse(predicate, if_true_do_, if_false_do_)
> v <- c(3,6,4,2,7,8,9,3)
> ifelse(v > 4, 3, 0)
[1] 0 3 0 0 3 3 3 0
> any(v > 4)
 [1] TRUE
> all(v > 4)
 [1] FALSE
```

Logical operators

`!` (NOT)

`&` or `&&` (AND)

`|` or `||` (OR)

R basics – Matrices

```
> matrix(1:12, nrow=3)    #default: byrow=F
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12

> matrix(1:12, nrow=3, byrow=T)
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
# In R, 'TRUE' is sometimes written as'T'
# and 'FALSE' as 'F'
```

R basics – Matrices

```
> matrix(1, nrow=3, ncol=5)
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    1    1    1    1
[2,]    1    1    1    1    1
[3,]    1    1    1    1    1

> diag(1:2, nrow=3, ncol=3)
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    2    0
[3,]    0    0    1
```

R basics – Matrices

```
> cbind(c(1,2,3),c(4,5,6))
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6

> rbind(c(1,2,3),c(4,5,6))
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

- Data conversion

```
> as.matrix(x)      # force x to be a matrix
```

R basics – Matrices

```
> M <- rbind(c(1,2,3),c(4,5,6))
> rbind(M, c(7,8,9))
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

R basics – Matrices

```
> M <- rbind(c(1,2,3,4),c(5,6,7,8))
> rbind(M, c(2,4,6))
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    2    4    6    2

Warning message:
In rbind(M, c(2,4,6)) :
  number of columns of result is not a multiple of vector length (arg 2)
```

R basics – Matrix indexing

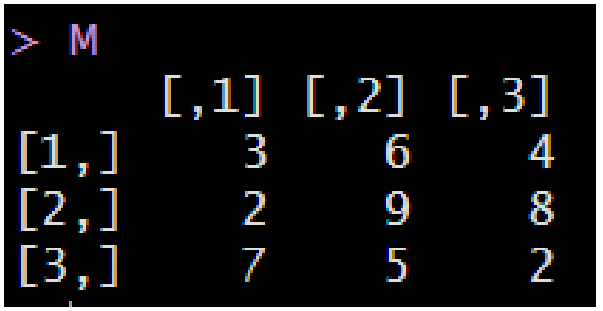
```
> M[i,]      # i-th row of matrix M
> M[,j]      # j-th column of matrix M
> M[i,j]     # [i,j] element of matrix M
```

```
> M <- rbind(c(3,6,4),c(2,9,8),c(7,5,2))
> M[2,3]
[1] 8
> M[1,]
[1] 3 6 4
> M[,3]
[1] 4 8 2
```

```
> M
     [,1] [,2] [,3]
[1,]    3    6    4
[2,]    2    9    8
[3,]    7    5    2
```

R basics – Matrix indexing

```
> M <- rbind(c(3,6,4),c(2,9,8),c(7,5,2))
> M[,c(1,3)]
     [,1] [,2]
[1,]    3    4
[2,]    2    8
[3,]    7    2
> M[c(2,3),3] <- 0
> M
     [,1] [,2] [,3]
[1,]    3    6    4
[2,]    2    9    0
[3,]    7    5    0
```



R basics – Matrix operations

```
> k * M      # multiply each element of
             # matrix M by scalar k

> A + B      # sum matrix A to matrix B

> A %*% B    # A × B cross-product
```

R basics – Lists

- Lists can be heterogeneous collection of objects

```
> x <- c(2,5,7)
> y <- c("a","b","c")
> z <- 3
> L <- list(x,y,z)
> L
[[1]]
[1] 2 5 7
[[2]]
[1] "a" "b" "c"
[[3]]
[1] 3
```

R basics – Lists

```
> names(L)      # we can assign names to lists,
[1] NULL        # but if we do not, list names
                # will be indexes and the names()
                # function will return NULL

> L <- list(A = c(2,4,6), B = c("a","b","c"))
> L
$A
[1] 2 4 6
$B
[1] "a" "b" "c"
```

R basics – List structure

```
# The $ symbol is used to select list attributes
> L$A          # List only elements of A attribute
[1] 2 4 6      # same for L[['A']]
> L['A']   # extract a sub-list
$A
[1] 2 4 6
> is.numeric(L$A)
[1] TRUE
> is.numeric( L['A'])
[1] FALSE
# While typeof(L$A) is "double" typeof(L['A'])
is still "list".
```

R basics – List structure

```
> str(L)     # Print L structure
$A: num [1:3] 1 2 3
$B: chr [1:3] "a" "b" "c"
> names(L)
[1] "A" "B"
> names(L) <- c("X","Y")     # change L names
> str(L)
$X: num [1:3] 1 2 3
$Y: chr [1:3] "a" "b" "c"
```

R basics – Data frames

- A data frame in R is a list of vectors having the same length, i.e. a collection of heterogeneous objects, where each attribute is a column.
- **Build data frames from lists:**

```
> L <- list(X = c(2,5,6), Y = c("a","b","c"))
> D <- data.frame(L)
> D
  X  Y
1 2 "a"
2 5 "b"
3 6 "c"
```

- List of attribute names of a data frame:

```
> names(D)        [1] "X" "Y"
```

R basics – Data frames

```
> str(D)
'data.frame': 3 obs. of 2 variables:
 $ X: num 2 5 6
 $ Y: chr  "a" "b" "c"

> summary(D) # Basic data frame statistics
 X                Y
 Min.   :2.000   Length:3
 1st Qu.:3.500   Class :character
 Median :5.000   Mode  :character
 Mean   :4.333
 3rd Qu.:5.500
 Max.   :6.000
```

R basics – Data frames

```
> data.frame(L) #only for matching dimensions
stringsAsFactors=FALSE has become the new default from R 4.0
# The data.frame() function used to convert
#attributes of type string to factors, i.e.
#categorical data, having values called levels.
#NOW instead data.frame(L) is equivalent to
data.frame(L, stringsAsFactors=FALSE)
#Otherwise:
> data.frame(L, stringsAsFactors=TRUE)
> str(D)
'data.frame': 3 obs. of 2 variables:
 $ X: num 2 5 6
 $ Y: Factor w/ 3 levels "a","b","c": 1 2 3
```

R basics – Data frames

- **Build data frames from matrices:**

```
> M <- rbind(c(3,6,4), c(2,9,8), c(7,5,1))
> D <- data.frame(M)
> D
  X1 X2 X3          # if no column name is given,
1  3  6  4          # default column names will
2  2  9  8          # be assigned (X1, X2, …)
3  7  5  1
> colnames(D) # equivalent to names(D)
[1] "X1" "X2" "X3"
> colnames(D) <- c("X","Y","Z")
> rownames(D)
[1] "1" "2" "3"
```

R basics – Data frames: read from file

- **Build data frames from text files:**

```
> D <- read.table(file [, options])
# options [default_value]
file              input tab-separated file (path)
header [F]        first row as a header
sep     [""] white space separator; "\t" for tab
quote   ["\"'"]   quoting characters
dec     ["."]     decimal separator
row.names     vector of row names
col.names     vector of column names
skip    [0]     skip n lines, before starting
fill    [T]     fill columns with different length
stringsAsFactors [F] change strings to factors
```

R basics – Data frames working examples

```
# Load metadata.txt file
> ?

# Example 1
# Find all the histone modification samples (as
# HM samples specified in the subtype column)
# from the Broad Institute (as broad elements
# in the lab column)
> ?
# Example 2
# Find all the filenames of samples coming from
# RAD21 transcription factor in females
# (antibody and sex columns)
> ?
```

R basics – Data frames working examples

```
# Example 3
# Print a table (use table() function) of cell
# lines (cell column) with their respective
# number of experiments not involving H3K27me3
# histone modification (antibody column)
> ?

# Note:  Use table() function that builds a
contingency table of the counts at each
combination of factor levels. Also, remember:
          AND is &          Equal is ==
          OR is |        Not equal is !=
          NOT is ! #
```

R basics – Data frames working examples

```
# Example 4
# Order metadata by cell line name (use order()
# function)
> ?

# Note:
# the function order() returns the ordered
# indexes of the vector, list, or dataframe
# that is in its argument
```

R basics – String manipulation



```
# Create strings from vectors/other strings
> paste(c(1,2,3), collapse="")
[1] "123"
> paste(c(1,2,3), collapse=" ")
[1] "1 2 3"
> stringr::str_sub(string='hi!!', start=3, end=4)
[1] "!!"
# Create vectors from strings
> strsplit("1;2;3", split =";")#returns a list
[[1]]
[1] "1" "2" "3"
> strsplit("1 2 3", " ")[[1]]#returns a vector
[1] "1" "2" "3"
```

R basics – Strings: a simple exercise

```
# Suppose having the following file path:
# /home/user1/documents/test.tab


# Change file extension from .tab to .txt
# using split and paste functions
# \\. Escape sequence to avoid problems with
dot in using strsplit function
```

R basics – Simple plots

- Scatter-plot:

```
> plot(x, y)

# Options
type      "p" for points; "l" for lines; etc.
main      plot title
sub       subtitle
xlab      x axis label
ylab      y axis label

# for more options type ?plot
```

R basics – Simple plots

- Histogram:

```
> hist(x)

# Options
breaks    a vector giving the breakpoints
          between histogram cells; a function
          to compute the vector of breakpoints;
          a single number giving the number of
          cells for the histogram; etc.
freq      if T, frequencies' plot (counts)
          if F, densities' plot (area sums to 1)
col       color used to fill bars

# for more options type ?hist
```

R basics – Simple plots

- Box-plot:

> **boxplot(formula)**

```
# Options
formula    e.g., y ~ x, where y is a numeric
           vector of data values to be splitted
           into groups according to the grouping
           variable x (usually a factor)
data       a data.frame (or list) from which the
           variables in formula should be taken
col        a vector of colors to fill each box

# for more options type ?boxplot
```

R basics – Simple plots: a working example

```
# Use data() to list sample datasets in R.
# We will use the Puromycin data set.
# This set is stored as a data frame with
# three attributes: conc, rate and state.
# Attribute conc is the concentration of
# substrate, rate is the  speed of the enzymatic
# reaction, and state is the state of a sample,
# (i.e. untreated or treated with Puromycin)
> head(Puromycin)    # have a look at the data frame

> summary(Puromycin)      # basic data frame stats
```

R basics – Simple plots: a working example

```
# Analyze the dataset making loess regressions
# and scatter plots for untreated and treated
# samples

# Split the dataset
> D1 <- Puromycin[Puromycin$state=="untreated",]
> D2 <- Puromycin[Puromycin$state=="treated",]
# loess model
> loess1 <- loess(D1$rate ~ D1$conc)
> loess2 <- loess(D2$rate ~ D2$conc)
# Model summary
> summary(loess1)
> summary(loess2)
```

R basics – Simple plots: export to PDF

```
# Scatter plots
# open a device for pdf file creation
> pdf("C:/Users/USER/Desktop/plot.pdf")

> par(mfrow=c(2,1))          # divide in 2 rows
                             # and 1 column

> plot(D1$conc, D1$rate)
> plot(D2$conc, D2$rate)

> dev.off()                  # close the device
```

R basics – Export/import data

```
# Export data to txt file
> write.table(VARIABLE, file = 'OUT_PATH.txt',
                quote = FALSE, sep = "\t")
# If quote=TRUE, any character or factor
# columns will be surrounded by double quotes.

# Save multiple data as R image (in .RData)
> save(VARIABLE(S), file = 'OUT_PATH.RData')
# use load('OUT_PATH.RData') to import saved data

# Use saveRDS(Onevar, file = "Onevar.rds")to save
one variable and OneVar <- readRDS(" Onevar.rds")
to import it in the current workspace
```

R basics – Functions

```
> myFname <- function(arg1, arg2, arg3=default, …)
  {
    body
  }

> myFname <- function(arg1, arg2, …) one_line_body

#In R, it is not necessary to include the return
#statement. When the function call is assigned to
#another variable, R can automatically return the
#value of whichever variable x is on the last line
#of the body of the function; still, return(x) can
#be used to print x value when it is not assigned
#to any other variable.
```



R basics – Control structures

```
> if(condition) {
      ...
  }


> if(condition) {
      ...
  } else if (condition) {
      ...
  } else {
      ...
  }
```

R basics – Iteration

```
> for(i in 1:length(v)){...}
> for(i in v){...}
> while(condition) {...}

> x<-c(2,5,7)
> for (i in 1:length(x)){print(i)}
[1] 1
[1] 2
[1] 3
> for (i in x){print(i)}
[1] 2
[1] 5
[1] 7
```

R basics – Iteration

```
# Apply a function to the margins of an array or
# matrix/numerical data frame returning a vector,
# array or list of values
> apply(X, APPLY_TO, FUNCTION)

# X         is an array or matrix
# APPLY_TO is 1 to apply the FUNCTION on rows
#           is 2 to apply the FUNCTION on columns…
# e.g.:
> apply(M, 2, mean)
# apply mean to columns of matrix M

# The apply function is fast and more concise than
# explicit loops.
```

R basics – Iteration

```
# Functions can be defined within apply
> apply(M, 2, function(x) SINGLE_LINE_BODY)

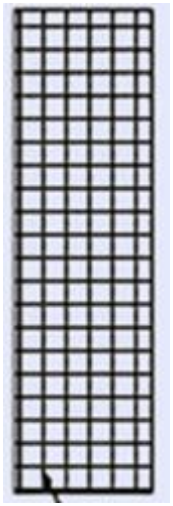
# e.g.:
> k <- 3
> apply(M, 2, function(x) all(x >= k))

# This command returns a Logical vector with TRUE,
# only for those columns from matrix M having all
# values greater or equal to the constant k
```

R scripting – EXERCISE_1

```
# Write an R function (first its pseudocode) to
# standardize a numeric matrix X wrt column values.
# The standardized version of X is:
# Z = (X - M) / S
# where M is the mean matrix of X and S the standard
# deviation matrix of X wrt column values.
# Use apply instead of a cycle.
#
# R functions that could be helpful:
#   MATRIX DIFFERENCE: -
#   MATRIX DIVISION: /
#   MATRIX CROSS-PRODUCT: %*%
#   TRANSPOSE: t(X)
```

X



R scripting – ANSWER_1

Supposing X has n rows and m columns,

```
standardizeData <- function(X) {
    U <- rep(1, nrow(X)) # vector
    M <- U %*% t(apply(X, 2, mean)) #[(n,1) %*% (1*m)]=(n,m)
    S <- U %*% t(apply(X, 2, sd))
    Z <- (X - M)/S
    return(Z)
}

 # matrix(U) is a single-column matrix(with n rows)
 # t(apply(X, 2, ---)) is a single-row matrix
   (with n columns)
```

R scripting – EXERCISE_2

```
# Write an R function (first pseudocode) to remove
# one of the two variables (columns) from an input
# matrix X with Pearson correlation coefficient
# greater or equal to a certain threshold value r
# (e.g. r=1).

# Use apply instead of a cycle.

# R functions that could be helpful:
# PEARSON CORRELATION: cor(X)
#    UPPER TRIANGULAR: upper.tri(X)
#       MAIN DIAGONAL: diag(X)
#      RETURN A VALUE: return(VAR)
```

R scripting – EXERCISE_2 – HINT

```
> X <- rbind(c(2,9,8,0,5,2),
             c(3,8,0,1,9,2),
             c(4,3,6,2,8,3))
```

variables / features

```
> X
     [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    2    9    8    0    5    2
[2,]    3    8    0    1    9    2
[3,]    4    3    6    2    8    3
```

statistical units / samples

```
> cor(X)
           [,1]       [,2]       [,3]      [,4]       [,5]      [,6]
[1,]  1.0000000 -0.9332565 -0.2401922 1.0000000  0.7205767 0.8660254
[2,] -0.9332565  1.0000000 -0.1245339 -0.9332565 -0.4234152 -0.9878292
[3,] -0.2401922 -0.1245339  1.0000000 -0.2401922 -0.8461538 0.2773501
[4,]  1.0000000 -0.9332565 -0.2401922 1.0000000  0.7205767 0.8660254
[5,]  0.7205767 -0.4234152 -0.8461538 0.7205767  1.0000000 0.2773501
[6,]  0.8660254 -0.9878292  0.2773501 0.8660254  0.2773501 1.0000000
```

R scripting – ANSWER_2

```
> R <- cor(X)
           [,1]       [,2]       [,3]       [,4]       [,5]       [,6]
[1,]  1.0000000 -0.9332565 -0.2401922  1.0000000  0.7205767  0.8660254
[2,] -0.9332565  1.0000000 -0.1245339 -0.9332565 -0.4234152 -0.9878292
[3,] -0.2401922 -0.1245339  1.0000000 -0.2401922 -0.8461538  0.2773501
[4,]  1.0000000 -0.9332565 -0.2401922  1.0000000  0.7205767  0.8660254
[5,]  0.7205767 -0.4234152 -0.8461538  0.7205767  1.0000000  0.2773501
[6,]  0.8660254 -0.9878292  0.2773501  0.8660254  0.2773501  1.0000000
```

```
> R[upper.tri(R)] <- 0
           [,1]       [,2]       [,3]       [,4]       [,5]       [,6]
[1,]  1.0000000          0          0          0          0          0
[2,] -0.9332565  1.0000000          0          0          0          0
[3,] -0.2401922 -0.1245339  1.0000000          0          0          0
[4,]  1.0000000 -0.9332565 -0.2401922  1.0000000          0          0
[5,]  0.7205767 -0.4234152 -0.8461538  0.7205767  1.0000000          0
[6,]  0.8660254 -0.9878292  0.2773501  0.8660254  0.2773501  1.0000000
```

```
# The correlation coefficient between variable #1 and #4 is 1. We want to remove
# only one of them (e.g. variable #1)
```

R scripting – ANSWER_2

```
> diag(R) <- 0
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
[1,]	0	0	0	0	0	0
[2,]	-0.9332565	0	0	0	0	0
[3,]	-0.2401922	-0.1245339	0	0	0	0
[4,]	1.0000000	-0.9332565	-0.2401922	0	0	0
[5,]	0.7205767	-0.4234152	-0.8461538	0.7205767	0	0
[6,]	0.8660254	-0.9878292	0.2773501	0.8660254	0.2773501	0

`apply(abs(R), 2, function(x) any(x == 1))` return columns having at least a 1-value

```
> X[, !apply(abs(R), 2, function(x) any(x == 1))]
```

	[,1]	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]		2	9	8	0	5	2
[2,]		3	8	0	1	9	2
[3,]		4	3	6	2	8	3

The original variable #1 has been filtered out

R scripting – ANSWER_2

```
removeCorr <- function(X, r=1) {
  R <- cor(X)
  R[upper.tri(R)] <- 0
  diag(R) <- 0
  NEW <- X[,!apply(abs(R),2,function(x) any(x >= r))]
  return(NEW)
}

# also - can be used instead of !
```

A first R script – mature expression data handling

```
# Define functions to handle mature expression data
removeCorr <- function(X, r=1) {
  R <- cor(X)
  R[upper.tri(R)] <- 0
  diag(R) <- 0
  NEW <- X[, !apply(R, 2, function(x) any(x >= r))]
  return(NEW)
}

standardizeData <- function(X) {
    U <- rep(1, nrow(X))
    M <- U %*% t(apply(X, 2, mean))
    S <- U %*% t(apply(X, 2, sd))
    Z <- (X - M)/S
    return(Z)
}
```

A first R script – mature expression data handling

```
# Standardize and filter out correlated vars

r <- max(cor(X))
R <- removeCorr(X, r)
Z <- standardizeData(R)

# Exporting to file

Z <- data.frame(Z)
write.table(Z, file = "result_first_script.txt", quote =
FALSE, sep = "\t")
```

R basics – Load/Run R code

- Load R code from a source file **`code.R`**

```
> source("/path/code.R")
```

- Launch a R script **`script.R`** from terminal

```
$ Rscript script.R [args]
```

OR

```
$ R CMD BATCH script.R [args]
```